

Mars Operations: Base Deployment (MOBD)

Relazione di progetto

Corso di *Metodi di Ottimizzazione per Big Data* — Prof. Andrea Cristofari
Università di Roma “Tor Vergata”, A.A. 2025/2026

Autori: Spiridigliozzi Davide, Rosi Federico.

Descrizione del problema

Il problema chiede di posizionare in modo ottimale $m = 1000$ moduli operativi $x^1, \dots, x^m$ su una mappa 2D di Marte. Nell'area sono già presenti una postazione fissa $z^0 = (0, 0)$ e quattro stazioni:

$$z^1 = (100, 100), z^2 = (-100, 100), z^3 = (-100, -100), z^4 = (100, -100).$$

Ogni modulo $x^i \in \mathbb{R}^2$ ha coordinate (x_1^i, x_2^i) , quindi il vettore delle variabili è

$$x = (x_1^1, x_2^1, x_1^2, x_2^2, \dots, x_1^{1000}, x_2^{1000})^\top \in \mathbb{R}^{2000}.$$

Vogliamo minimizzare il costo totale, somma di quattro contributi:

$$\min_{x \in \mathbb{R}^{2000}} f(x) = f_1(x) + f_2(x) + f_3(x) + f_4(x).$$

I quattro termini sono:

- f_1 — **dispersione della rete.** $f_1(x) = 0.1 \sum_{i < j} \left(1 - e^{-\|x^i - x^j\|^2} \right)$. Misura

la perdita di coesione della rete: vale quasi 0 quando i moduli sono vicini e tende a crescere quando si allontanano. La somma è su tutte le $\binom{1000}{2} = 499\,500$ coppie.

- f_2 — **distanza dalle stazioni fisse.** $f_2(x) = \frac{1}{m} \sum_{i=1}^m \sum_{q=0}^4 \|x^i - z^q\|^2$. È

una penalità quadratica che tiene i moduli vicini alle 5 stazioni fisse.

- f_3 — **deviazione dalla distanza operativa ideale.**

$$f_3(x) = 1000 \sum_{i=1}^m \left(\frac{\log(1 + \|x^i - s^i\|^2)}{\log(1 + r^2)} - 1 \right)^2, \quad r = 100,$$

dove $s^i = ((-1)^i i 0.2, (-1)^i i 0.2)$ è il punto di riferimento del modulo i . Vale 0 esattamente quando $\|x^i - s^i\| = r = 100$ per ogni i .

- f_4 — **costo ambientale (black-box)**. Modella fenomeni non ideali (ombreggiamento radio, polvere, interferenze, ecc.). Non ha una formula nota: si può solo valutare con il simulatore `mobd` (opzione `-b`), che è deterministico e leggi in input il file `x.txt` con i 2000 valori.

Metodo scelto e richiami teorici

Il problema è un'ottimizzazione **non vincolata** in $\mathbb{R}^n$ con $n = 2000$. Richiamiamo prima le condizioni di ottimalità che useremo come riferimento.

Condizioni necessarie del primo ordine. Se x^* è un minimo locale e f è differenziabile in x^* , allora $\nabla f(x^*) = 0$ (punto stazionario).

Condizioni del secondo ordine. Se x^* è un minimo locale e f è due volte differenziabile, allora $\nabla f(x^*) = 0$ e $\nabla^2 f(x^*) \succeq 0$; viceversa, se $\nabla f(x^*) = 0$ e $\nabla^2 f(x^*) \succ 0$ allora x^* è un minimo locale stretto.

Visto che f non è convessa (per via di f_1 e di f_4), possiamo puntare in generale a un punto stazionario, non al minimo globale.

Block Coordinate Descent (BCD) e versione randomizzata

Il problema ha una struttura a blocchi naturale: il vettore x si divide in $m = 1000$ blocchi da 2 componenti ciascuno (le due coordinate di un modulo). Questo ci ha portato a usare il **Block Coordinate Descent**: a ogni passo aggiorniamo un solo blocco i , lasciando fermi gli altri, con un passo di discesa lungo il gradiente parziale:

$$x_{k+1}^i = x_k^i - \alpha_k \nabla_{x^i} f(x_k).$$

Usiamo la variante **Gauss-Seidel**: i blocchi già aggiornati nell'epoca corrente vengono usati subito nei calcoli successivi (a differenza della variante Jacobi, che userebbe sempre le posizioni di inizio epoca). Inoltre, a ogni epoca aggiorniamo i blocchi seguendo una **permutazione casuale** degli indici (da qui "Randomized" BCD): questo evita che un ordine fisso introduca comportamenti sistematici e in pratica aiuta la convergenza.

Passo decrescente

Usiamo un passo (stepsize) che diminuisce con le epoche:

$$\alpha_k = \frac{\alpha_0}{(1 + \beta k)^\gamma},$$

con $\alpha_0 > 0$ passo iniziale, $\beta > 0$ velocità di decrescita e $\gamma \in (0, 1]$ esponente.

Teorema (convergenza con passo decrescente). Sia f continuamente differenziabile con gradiente Lipschitz, e sia $\{x_k\}$ la successione generata dal RBCD Gauss-Seidel con passo che soddisfa

$$\lim_{k \rightarrow \infty} \alpha_k = 0 \text{ e } \sum_{k=0}^{\infty} \alpha_k = +\infty.$$

Allora ogni punto di accumulazione di $\{x_k\}$ è un punto stazionario di f .

Per il nostro schema, $\alpha_k \rightarrow 0$ vale per ogni $\gamma > 0$, mentre $\sum_k \alpha_k = +\infty$ vale

se e solo se $\gamma \leq 1$. Scegliendo $\gamma = 0.6$ entrambe le condizioni sono soddisfatte.

Gradiente di f_4 con le differenze finite

Per f_1, f_2, f_3 conosciamo il gradiente in forma analitica (vedi sotto). Per f_4 , che è una black-box, lo approssimiamo con le **differenze finite in avanti**:

$$\frac{\partial f_4}{\partial x_j^i}(x) \approx \frac{f_4(x + He_{2i+j}) - f_4(x)}{H}, \quad j \in \{0, 1\},$$

con passo $H = 10^{-4}$ ed e_k il k -esimo vettore della base canonica. Servono quindi $2m = 2000$ valutazioni del simulatore per epoca (più una per il valore base). Il valore di H è un compromesso: troppo grande dà errore di troncamento, troppo piccolo dà cancellazione numerica; 10^{-4} funziona bene per le scale del problema (coordinate dell'ordine di 10^1 – 10^2).

Problemi incontrati e soluzioni

Mettere in pratica l'algoritmo ha richiesto di risolvere alcuni problemi.

1. f_4 è lentissima. Ogni chiamata al simulatore costa circa 5 secondi: in modo sequenziale, 2000 chiamate a epoca farebbero quasi 3 ore a epoca. Poiché le 2000 perturbazioni sono indipendenti tra loro, le calcoliamo in parallelo con ThreadPoolExecutor: con N thread il tempo scende a circa $\lceil 2000/N \rceil \times 5$ s a epoca.

2. Base coerente per le differenze finite. Nella variante Gauss-Seidel x cambia durante l'epoca, quindi calcolare il gradiente di f_4 "al volo" userebbe un valore base incoerente. Abbiamo adottato una strategia **stale-base**: all'inizio di ogni epoca fissiamo uno snapshot $\hat{x}$ e calcoliamo $\hat{f}_4 = f_4(\hat{x})$ una volta sola;

tutte le 2000 perturbazioni usano la stessa base $\hat{x}$. Il gradiente di f_4 risulta leggermente “vecchio” rispetto allo stato di fine epoca, ma con passo piccolo è accettabile.

3. Inizializzazione. Partire da posizioni casuali porta spesso a f_3 molto grande, che domina il gradiente e spinge l’algoritmo lontano da buone soluzioni. Usiamo allora un **warm start** costruito in modo che $f_3 = 0$ all’inizio: mettiamo ogni modulo esattamente a distanza $r = 100$ dal suo riferimento s^i ,

$$x_0^i = s^i + r \frac{-s^i}{\|s^i\|},$$

così $\|x_0^i - s^i\| = r$ e quindi $f_3 = 0$.

4. I/O su disco. Ogni chiamata al simulatore scrive un file da 2000 righe; con migliaia di scritture a epoca il disco diventa un collo di bottiglia. Su Linux scriviamo i file temporanei su `/dev/shm` (un filesystem in RAM), molto più veloce.

5. Costo di f_1 . Il gradiente di f_1 richiede una somma su tutti gli altri moduli, quindi $O(m^2)$ per epoca. Per renderlo gestibile usiamo operazioni vettorizzate NumPy (broadcasting ed `einsum`), che girano in C ed evitano i cicli Python espliciti.

Come eseguire il codice

Modalità completa (quella che usiamo per la competizione):

```
python mobd_optimizer.py --mode full --full-epochs 20
```

Modalità veloce (solo gradienti analitici, utile per provare):

```
python mobd_optimizer.py --mode fast --fast-epochs 100
```

Verifica della soluzione con il simulatore:

```
./linux/mobd x.txt -b # solo f4
```

I parametri principali sono `--alpha0`, `--beta`, `--gamma` (passo), `--seed` (seme casuale), `--workers` (numero di thread per f_4) e `--no-resume` (riparte dal warm start invece di proseguire da un `x.txt` esistente).